Lab 12 — Questions

Question 1 [Understanding]

An `api` Deployment with three replicas was created, then updated twice. How many ReplicaSets exist, how many Pods, and what does `kubectl get rs` show? Why are the old ReplicaSets not deleted, and which setting bounds their number?

Question 2 [Diagnosis]

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: backend
```

`kubectl apply` rejects this Deployment. Give the error message in substance, explain the rule it breaks, and say what would happen if the cluster accepted it.

Question 3 [Prediction]

A three-replica Deployment with the default strategy moves from `api:1.0` to `api:1.1`. Describe the sequence of `ScalingReplicaSet` Events (how many Pods of each version at each step), and explain why the cluster does not simply create three new Pods at once.

Question 4 [Diagnosis]

```
$ kubectl set image deployment/api api=localhost/api:bad
$ kubectl get pods
NAME                   READY   STATUS    RESTARTS      AGE
api-6c7bb54fc5-c2lhj   1/1     Running   0             32s
api-6c7bb54fc5-km2h6   1/1     Running   0             40s
api-6c7bb54fc5-s9c8f   1/1     Running   0             40s
api-6c87f96dd5-4v6s6   0/1     Error     2 (24s ago)   25s
```

Why is there only one Pod of the new version, why are the three old ones still running, and what does `kubectl rollout status` answer? Give the two commands that reveal the cause and then repair.

Question 5 [Analysis]

A colleague sets a `livenessProbe` that queries `/actuator/health`, which returns `DOWN` as soon as the database is unreachable. The database goes down for five minutes. Describe what happens to the API, why it is worse than having no probe, and what should have been done.

Question 6 [Prediction]

In a Deployment with three healthy replicas, you change the template so that the `readinessProbe` targets port `8081` (where nothing listens). What do `kubectl get pods` and `kubectl get deployment api` show thirty seconds later? Does the service still answer? Same question if it is the `livenessProbe` you had broken.

Question 7 [Understanding]

Explain what `CrashLoopBackOff` is: is it a state of the application, of the container or of the kubelet? Why do the restarts spread out? Which command shows the cause, and why can `kubectl logs` without options show nothing useful?

Question 8 [Analysis]

Compare the `HEALTHCHECK` of lab 10 with an `httpGet` `readinessProbe`: who runs the check, where must the tool (`wget`, `curl`) be, what effect does a failure have, and which of the two works with a distroless image?

Question 9 [Diagnosis]

`kubectl rollout history deployment/api` shows revisions `2, 4, 6, 7`: numbers `1, 3, 5` are missing. Explain where those gaps come from (hint: every `rollout undo` creates a revision) and why it is not a sign of trouble.

Question 10 [Analysis]

One Deployment has `maxSurge: 0` and `maxUnavailable: 1`; another has `maxSurge: 1` and `maxUnavailable: 0`. For three replicas, describe the update in each case (number of Pods at the peak and at the trough), and say which suits a cluster whose nodes are full and which suits a service that must never lose capacity.

Question 11 [Diagnosis]

After `kubectl scale deployment api --replicas=5`, two Pods stay `Pending` indefinitely while the other three are `Running`. What is the most likely cause on a single-node k3s, where do you read it, and what happens if you apply a `rollout` in that state?

Question 12 [Analysis]

A developer bypasses Kubernetes: they `kubectl exec` into a Pod, edit a configuration file and restart the process by hand. Describe what happens at the next rollout, at the next restart of the Pod, and on the other two replicas. What should they have done (two objects from the upcoming labs)?

Question 13 [Analysis]

A lone `Pod` written in a manifest (`kind: Pod`) is applied in production "because we only have one instance". Give four things this choice prevents, compared with a Deployment at `replicas: 1`, and the only case where a bare Pod is the right object.